

Modul 7 Semaphore

Tujuan Praktikum

1. Praktikan mampu mengimplementasikan semaphore.
2. Praktikan mampu mengimplementasikan mutex.

7.1 Operasi Semaphore

Terdapat 3 operasi yang bisa dilakukan pada semaphore yaitu: inisiasi, signal dan wait.

7.1.1 Inisiasi

Pada Xinu, inisiasi semaphore dilakukan sebagai berikut:

`nama_semaphore = semcreate(nilai_inisiasi_semaphore)`

Contoh kode:

```
sid32 s1,s2;           // variabel s1 dan s2 dengan tipe sid32
s1 = semcreate(3);      // inisiasi s1 dengan nilai 3
s2 = semcreate(1);      // inisiasi s2 dengan nilai 1

resume(create(sndA, 1024, 20, "process A", 1, s1, s2)); //proses A menggunakan 2 semaphore
resume(create(sndB, 1024, 20, "process B", 1, s1)); // proses B hanya butuh 1 semaphore
```

7.1.2 Signal

Signal berarti increment nilai semaphore. Pada Xinu, semaphore signal menggunakan sintak sebagai berikut:

`signal(nama_semaphore)`

Contoh kode:

```
void sndB(sid32 s1)
{
    ... kode sebelumnya. // nilai s1 = 3

    signal(s1); //increment nilai s1; jika nilai s1 sebelum dieksekusi adalah 3 maka setelah signal
    dieksekusi maka nilai s1 menjadi 4

    ...kode sesudahnya // sekarang nilai s1 = 4
}
```

7.1.3 Wait

Wait berarti decrement nilai semaphore. Jika bernilai negatif maka proses yang memanggil wait akan block (perintah setelah wait akan dieksekusi setelah proses diunblock). Pada Xinu, semaphore wait menggunakan sintak sebagai berikut:

`wait(nama_semaphore)`

Contoh kode:

```
void sndA(sid32 s1, sid32 s2)
{
  ... kode sebelumnya. // misalkan nilai s1 = 3; s2 = 0

  wait(s1); // nilai s1 sekarang menjadi 2
  print ("A"); // perintah ini akan dieksekusi
  wait(s2); // nilai s2 akan menjadi -1 karena negatif maka proses A akan diblock
  print("B"); // perintah ini akan dieksekusi setelah proses A diunblock oleh proses lain

  ...kode sesudahnya //
}
```

7.2 Pola Pada Semaphore

Pola-pola yang sering digunakan dalam semaphore adalah **signaling** dan **mutex**.

7.2.1 Signaling

Pola signaling digunakan sebagai cara untuk **memastikan** bahwa suatu proses awal (P1) telah selesai dan baru proses selanjutnya (P2) bisa dijalankan.

Berikut adalah pseudo codenya:

Inisiasi semaphore:

s1 = 1 (P1 harus berjalan dahulu sehingga diinisiasi dengan 1).

s2 = 0 (P2 berjalan setelah P1 sehingga harus menunggu terlebih dahulu dan diinisiasi dengan nilai 0).

Pola semaphore:

Tabel 7-1 Pola Signaling

P1	P2
wait(s1)	wait(s2)
//P1 melakukan sesuatu	//P2 melakukan sesuatu
signal(s2)	signal(s1)

Perhatikan letak signal(s1), signal(s2), wait(s1) dan wait(s2)! Letaknya harus mengikuti pola di atas untuk memastikan signaling terjadi.

Contoh kode:

```
sndA(sid32 s1, sid32 s2){

  wait(s1);
  print ("A");
  signal(s2);
```

```

}
sndB(sid32 s1, sid32 s2){

wait(s2);
print ("B");
signal(s1);

}

```

Memastikan "A" diprint sebelum "B".

7.2.2 Mutex

Pola mutex digunakan untuk memastikan bahwa suatu variable (resource) hanya dapat diakses oleh satu proses saja.

Inisiasi mutex:

sem_1 = 1 (mutex selalu diinisiasi dengan 1. Jika ingin suatu resource bisa diakses secara bersamaan oleh n proses maka mutex diinisiasi dengan n).

Contoh kode:

```

sid32 mutex;
m1 = semcreate(1); // mutex selalu diinisiasi dengan 1

```

Pola mutex:

Tabel 7-2 Pola Mutex

P1	P2
wait(m1)	wait(m1)
//Critical section, hanya boleh 1 proses akses	//Critical section, hanya boleh 1 proses akses
signal(m1)	signal(m1)

Jika terdapat P3, pola tetap sama yaitu wait(m1); critical section; signal(m1)

P3
wait(m1)
//Critical section, hanya boleh 1 proses akses
signal(m1)

Contoh kode:

```
sndA(sid32 m1)
{
    wait(m1);
    saldo = saldo + setor; //critical section
    signal(m1);
}

sndB(sid32 m1)
{
    wait(m1);
    saldo = saldo + setor; //critical section
    signal(m1);
}
```



Fakultas Informatika
School of Computing
Telkom University

